Lecture 3: CPU Organization - Datapath & Control Unit

1. Von Neumann vs. Harvard Architectures

The way a computer's CPU interacts with its memory is a fundamental design decision. Two primary architectures define this interaction:

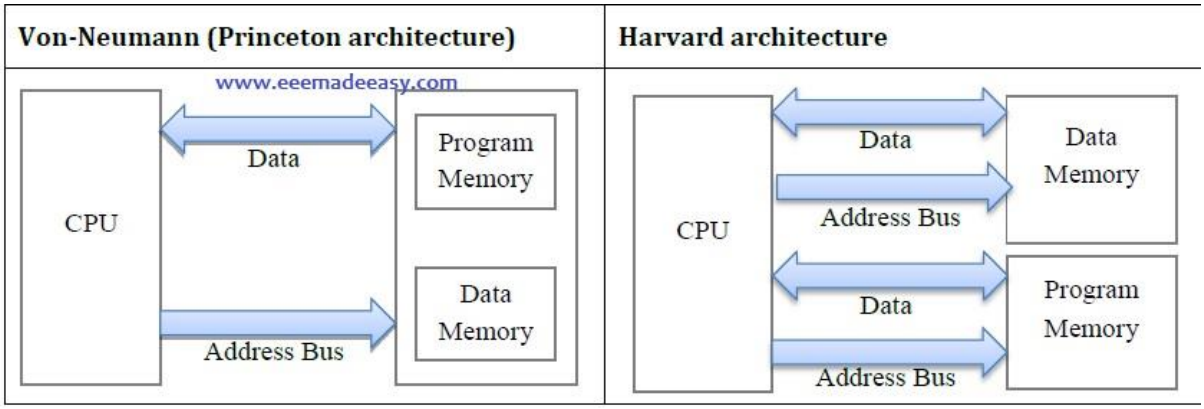
1.1. Von Neumann Architecture (Stored Program Concept)

- **Concept:** This architecture, proposed by John von Neumann in the 1940s, is the most common design for modern general-purpose computers. Its defining characteristic is that **both program instructions and data are stored in a single, shared memory space.**
- **Single Bus:** The CPU uses a **single address and data bus** to access both instructions and data from memory.
- **Sequential Access:** Since instructions and data share the same bus, they cannot be fetched simultaneously. The CPU must fetch an instruction, then fetch data (if needed for that instruction), leading to a bottleneck.
- **Flexibility:** Highly flexible because the same memory space can be used for instructions and data. This allows for dynamic allocation and self-modifying code (though the latter is generally discouraged in modern programming practices due to security and complexity).
- **"Von Neumann Bottleneck":** The shared bus creates a bottleneck as the CPU can only perform one memory access (fetch instruction OR fetch data) at a time, limiting the speed of computation.
- **Examples:** Most general-purpose computers (PCs, servers, smartphones) use a modified Von Neumann architecture.

1.2. Harvard Architecture

- **Concept:** Originated from the Harvard Mark I computer. In this architecture, **separate memory spaces and separate buses are used for instructions and data.**
- **Dual Buses:** The CPU has **one address and data bus for instruction memory** and a **separate address and data bus for data memory.**
- **Parallel Access:** This allows the CPU to fetch an instruction and fetch/store data *simultaneously*, significantly improving performance and throughput.
- **Less Flexible:** Less flexible than Von Neumann because the memory spaces are distinct. It's harder to use data memory for instructions or vice versa.

- **No "Bottleneck":** Eliminates the Von Neumann bottleneck by allowing concurrent instruction and data fetches.
- **Examples:** Often found in embedded systems, Digital Signal Processors (DSPs), and microcontrollers where speed and predictable performance are critical (e.g., in audio/video processing, automotive systems). Modern CPUs also implement aspects of Harvard architecture internally (e.g., separate L1 instruction and data caches) to mitigate the Von Neumann bottleneck, even if the external view is still Von Neumann.



Feature	Von Neumann Architecture	Harvard Architecture
Memory	Single shared memory for instructions & data	Separate memories for instructions & data
Buses	Single bus for both	Separate buses for instructions & data
Access	Sequential (instruction OR data)	Parallel (instruction AND data)
Bottleneck	"Von Neumann Bottleneck" present	No "Von Neumann Bottleneck"
Flexibility	High (self-modifying code possible)	Lower (dedicated memory spaces)
Typical Use	General-purpose computers, PCs	Embedded systems, DSPs, microcontrollers

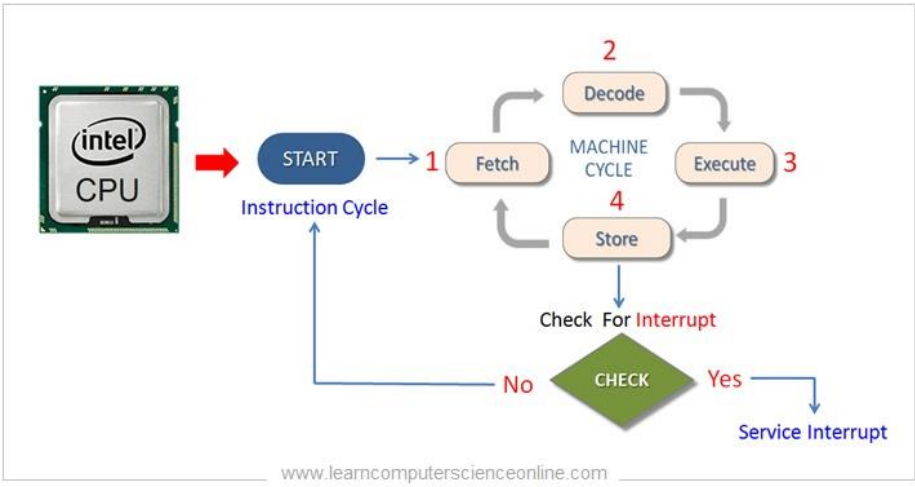
2. The Fetch-Decode-Execute Cycle (The CPU's Core Loop)

The CPU operates by continuously repeating a fundamental sequence of steps known as the **Fetch-Decode-Execute Cycle** (also known as the Instruction Cycle). This cycle describes how the CPU processes each instruction from a program.

2.1. Steps of the Cycle:

1. **Fetch (Instruction Fetch - IF):**
    - The CPU retrieves the next instruction from memory.
    - The **Program Counter (PC)** holds the memory address of the instruction to be fetched.
    - The PC's content is transferred to the **Memory Address Register (MAR)**.
    - The Control Unit then sends a "read" signal to memory.
    - The instruction at that address is retrieved from memory and placed into the **Memory Data Register (MDR)** (or directly into the Instruction Register).
    - Finally, the instruction is transferred from the MDR to the **Instruction Register (IR)**.
    - The PC is then incremented to point to the next instruction in sequence.
2. **Decode (Instruction Decode - ID):**
    - The Control Unit (CU) interprets the instruction that is now in the IR.
    - It determines what operation needs to be performed (e.g., ADD, LOAD, JUMP) and identifies the operands (data) involved.
    - This involves breaking down the instruction's opcode and operand fields.
    - Based on the decoded instruction, the CU generates the necessary control signals to activate the appropriate components in the Datapath for the next step.
3. **Execute (Execution - EX):**
    - The actual operation specified by the instruction is performed.
    - If it's an arithmetic or logical operation, the **ALU** performs the calculation.
    - If it's a data transfer operation, data is moved between registers, or between registers and memory (using MAR and MDR).
    - If it's a control flow instruction (e.g., JUMP, BRANCH), the PC might be updated to a new address, altering the flow of the program.

- Results are typically stored in registers or written back to memory.



2.2. Interplay:

These three stages work in a continuous loop. Once an instruction is executed, the cycle begins again with fetching the next instruction (which the PC now points to).

3. Datapath: The Data-Handling Components

The Datapath consists of the hardware components that hold and manipulate data under the direction of the Control Unit. It includes registers, the ALU, and the internal buses.

3.1. Registers

Registers are small, high-speed storage locations directly accessible by the CPU. They are faster than main memory and are used to temporarily hold data, instructions, and addresses during processing.

- **General-Purpose Registers (GPRs):**
  - Used by programmers to store arbitrary data values during computation.
  - The number and size (e.g., 32-bit, 64-bit) of GPRs vary between different CPU architectures (e.g., x86 typically has fewer GPRs than ARM).
  - Examples: AX, BX, CX, DX in x86 architecture (though often used for specific purposes like accumulator, base, count, data respectively). In ARM, R0 to R12 are general-purpose.
- **Special-Purpose Registers:** These registers have dedicated functions and are crucial for the CPU's operation.

- **Program Counter (PC):**
  - Holds the **memory address of the NEXT instruction to be fetched.**
  - Also called **Instruction Pointer (IP)** in some architectures.
  - It is automatically incremented after each instruction fetch, ensuring sequential program execution. JUMP or BRANCH instructions modify the PC to change the flow.
- **Instruction Register (IR):**
  - Holds the **instruction currently being executed.**
  - The fetched instruction is loaded into the IR for decoding by the Control Unit.
- **Memory Address Register (MAR):**
  - Holds the **memory address of the data or instruction to be accessed.**
  - When the CPU needs to read from or write to memory, the desired address is loaded into the MAR.
- **Memory Data Register (MDR):**
  - Holds the **data being transferred to or from memory.**
  - If the CPU is reading from memory, the data retrieved is placed in the MDR.
  - If the CPU is writing to memory, the data to be written is placed in the MDR.
  - Also known as **Memory Buffer Register (MBR).**
- **Status Register / Program Status Word (PSW) / Flags Register:**
  - Contains condition codes or flags (e.g., Zero, Carry, Negative, Overflow) set by ALU operations.
  - Used by conditional branch instructions to make decisions (e.g., "jump if the previous result was zero").

3.2. Arithmetic Logic Unit (ALU)

- As discussed in Week 2, the ALU is a combinational circuit responsible for performing all arithmetic operations (addition, subtraction, etc.) and logical operations (AND, OR, NOT, XOR) on data.

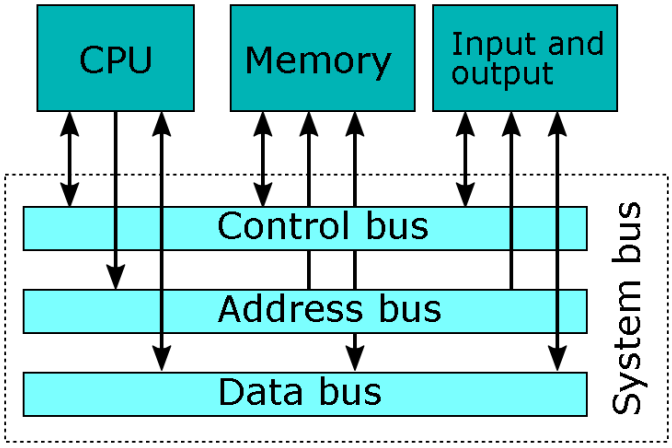
- It takes operands from registers (or MDR) and outputs results back to registers (or MDR), also setting status flags.

3.3. Buses

Buses are collections of electrical pathways that allow different components within the computer system to communicate and transfer data.

- **Data Bus:**
  - Carries the actual **data** being transferred between the CPU, memory, and I/O devices.
  - It is **bi-directional**, meaning data can flow in both directions (e.g., CPU to memory, or memory to CPU).
  - Its width (e.g., 32-bit, 64-bit) determines how many bits can be transferred simultaneously, impacting performance.
- **Address Bus:**
  - Carries the **memory addresses** (or I/O port addresses) that the CPU wants to access.
  - It is **uni-directional** (CPU to memory/I/O). The CPU puts an address on the bus, and memory/I/O components decode it to locate the specific data.
  - Its width determines the maximum amount of physical memory the CPU can address (2width). E.g., a 32-bit address bus can address 232 bytes (4 Gigabytes).
- **Control Bus:**
  - Carries **control signals** from the Control Unit to other components, coordinating operations.
  - It is **bi-directional**.
  - Examples of control signals:
    - **Read/Write:** Indicates whether the CPU wants to read from or write to memory/I/O.
    - **Memory Request:** Signals a memory operation.
    - **I/O Request:** Signals an I/O operation.
    - **Clock:** Synchronizes operations across the system.
    - **Interrupt Request:** Signals that an I/O device needs attention.

- **Bus Grant/Request:** For managing bus access when multiple devices want to use it.



4. Control Unit: The Coordinator

The Control Unit (CU) is the "manager" or "orchestrator" of the CPU. It interprets instructions and generates the necessary control signals to activate the various components of the Datapath and manage the flow of data. It ensures that operations occur in the correct sequence and at the right time.

4.1. Hardwired Control

- **Concept:** The control signals are generated by a fixed, physical logic circuit (combinational logic gates, decoders, encoders, etc.).
- **Design:** The logic is directly implemented in hardware.
- **Speed:** Very fast because the control signals are generated directly by logic circuits.
- **Flexibility:** Not flexible. To change the instruction set or add new instructions, the hardware logic must be redesigned, which is complex and expensive.
- **Typical Use:** Used in simpler or high-performance RISC processors where the instruction set is fixed and optimized for speed.

4.2. Microprogrammed Control (Conceptual)

- **Concept:** The control signals are generated by a sequence of **microinstructions** stored in a special control memory (ROM or EPROM) within the Control Unit.
- **Design:** Each instruction in the computer's instruction set (e.g., "ADD R1, R2") is broken down into a series of simpler, atomic operations called "micro-operations" (e.g., "transfer R1 to ALU input A," "transfer R2 to ALU input B," "ALU perform ADD," "store ALU result in R1"). This sequence of micro-operations forms a microprogram.
- **Speed:** Slower than hardwired control because each microinstruction needs to be fetched and executed.
- **Flexibility:** Highly flexible. To change the instruction set or add new instructions, the microprogram in the control memory can be updated (like programming a tiny computer within the CU). This is much easier than redesigning hardware.
- **Typical Use:** Used in more complex CISC processors where the instruction set is large and might evolve, or where debugging complex instructions is easier with microcode.

Feature	Hardwired Control	Microprogrammed Control

Logic	Implemented directly in combinational logic	Stored as microinstructions in control memory
Speed	Faster	Slower
Flexibility	Less flexible (hardware change needed)	More flexible (firmware update possible)
Complexity	Complex for large instruction sets	Simpler to design for complex ISAs
Typical Use	RISC, simple processors	CISC, complex processors

5. Introduction to Simple Instruction Execution Flow

Let's trace a very simple hypothetical instruction to see how the Datapath and Control Unit work together.

Hypothetical Instruction: ADD R1, R2 (Add the content of Register R2 to Register R1, store result in R1)

Assume:

- PC currently holds the address of the ADD R1, R2 instruction.
- R1 and R2 are General-Purpose Registers.
- The system has an ALU, MAR, MDR, and IR.

Execution Steps (Simplified):

1. **Fetch Cycle:**
    - **CU Action:** Places contents of PC into MAR.
    - **Datapath Action:** MAR now holds the address of ADD R1, R2.
    - **CU Action:** Sends "Read" signal to Memory.
    - **Datapath Action:** Memory reads the instruction at MAR's address and places it into MDR.
    - **CU Action:** Transfers content of MDR to IR.
    - **Datapath Action:** IR now holds the ADD R1, R2 instruction.
    - **CU Action:** Increments PC to point to the next instruction.
2. **Decode Cycle:**
    - **CU Action:** Decodes the ADD R1, R2 instruction from the IR.
    - **CU Action:** Determines that this is an ADD operation and involves registers R1 and R2.
    - **CU Action:** Prepares control signals for the execution phase (e.g., activate ALU's ADD function, route R1 and R2 to ALU inputs).
3. **Execute Cycle:**
    - **CU Action:** Sends control signals to move content of R1 to ALU input A.
    - **Datapath Action:** R1's value is placed on an internal bus, routed to ALU input A.
    - **CU Action:** Sends control signals to move content of R2 to ALU input B.

- **Datapath Action:** R2's value is placed on an internal bus, routed to ALU input B.
- **CU Action:** Sends control signals to the ALU to perform the ADD operation.
- **Datapath Action:** ALU adds the values from its inputs.
- **Datapath Action:** ALU produces the sum and sets status flags.
- **CU Action:** Sends control signals to store the ALU's result back into R1.
- **Datapath Action:** The ALU's result is placed on an internal bus and written into R1.

4. **Repeat:** The cycle then restarts, fetching the instruction pointed to by the new PC value.

This simple example illustrates the coordinated action of the Control Unit and the Datapath elements in carrying out a single instruction. All CPU operations, no matter how complex, are ultimately broken down into these fundamental steps involving data movement and manipulation orchestrated by the Control Unit.